

Fleet on the Floor: A Fog-to-Cloud Pipeline for Robotics Health Monitoring

Goutham Uppu

Student ID: X25167936

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

x25167936@student.ncirl.ie

Abstract—Warehouse Robotics Fleet Monitoring instruments a ten-robot autonomous mobile robot (AMR) fleet working two warehouse zones, capturing five telemetry channels per robot (battery level, payload weight, motor temperature, position drift and task-queue depth) through a fog gateway that windows, aggregates and threshold-checks the readings before batching them onward to a cloud tier built from Amazon SQS, two AWS Lambda functions, DynamoDB and API Gateway. A pre-deployment code audit found and corrected four defects: a DynamoDB scan-pagination undercount, a fog publisher that sent one SQS message at a time instead of exploiting the batch API, a credential-gating fault present in all three AWS-facing classes that would have crashed the fog node outright on startup outside a local emulator, and a project-documentation claim that overstated how much of the pipeline architecture was invented here rather than built on established, published patterns. The corrected stack was then provisioned to a real AWS Academy account through a single Terraform apply and independently verified end to end: 116 JUnit tests pass across four Maven modules, the health endpoint reports all four pipeline checks true, and a real-browser check confirmed the dashboard renders live fleet data with no meaningful console errors.

Keywords—fog computing, autonomous mobile robots, warehouse automation, Amazon SQS, AWS Lambda, DynamoDB, Infrastructure as Code, Terraform, fleet health monitoring

I. INTRODUCTION

Autonomous mobile robots (AMRs) have moved from pilot projects to a standard fixture on warehouse floors, ferrying totes and pallets between racking, picking stations and dock doors without a fixed rail to follow. A shop-floor fleet-management system built for exactly this setting found that dispatch decisions improve once a robot's position, battery state and current task load are known live, not just its nominal schedule [1]. A recent survey of intralogistics AMR deployments reaches a related conclusion from the opposite direction: it names resource scheduling, power consumption and safety as gaps the field has not closed, in large part because so few deployments instrument the robots closely enough to reason about all three together [2]. Warehouse Robotics Fleet Monitoring targets that gap directly for a fleet of ten AMRs split across two zones, zone-a and zone-b, giving each robot five independent telemetry channels (battery level, payload weight, motor temperature, position drift, and task-queue depth) and a pipeline that turns those channels into a live operational picture instead of a log read after a breakdown.

Bonomi et al. define fog computing as an extension of the cloud paradigm out to the edge of the network, placing storage, compute and networking on nodes that sit physically close to where data originates [3]. That definition describes this project's own topology closely: a fog gateway runs on the same local segment as the ten sensor containers, windowing their readings and evaluating fleet-health thresholds before anything leaves the building. The cloud tier that eventually receives the aggregated result needs its own definition, and the NIST reference model supplies one: on-demand, broadly accessible, elastically provisioned services metered by use [4]. DynamoDB, SQS, Lambda and API Gateway all satisfy that description, and the boundary between the two tiers is treated as a design decision: what crosses it is a windowed aggregate, not a raw sample, because the fog node exists specifically to make that reduction before anything crosses the network.

The project's objective was to build and deploy a complete fog-to-cloud pipeline for this fleet, not to simulate one against a local emulator and stop there. Ten sensor containers, five sensor types across two zones, generate readings on independent sample and dispatch intervals; a fog gateway ingests, buffers and windows them per robot and per zone, evaluates a set of alert rules against configurable thresholds, and relays the aggregates onward in batches; a queue decouples the fog tier from the cloud tier so that neither side blocks on the other; a Lambda function consumes the queue into a DynamoDB table; and a dashboard, backed by a second Lambda function behind a real API Gateway, renders a fleet-ops view for whoever is watching the floor.

Meeting that objective set a specific bar: a believable multi-sensor simulation with bounded, physically plausible drift instead of uniform random noise; a fog tier that does real aggregation work (windowing, threshold evaluation, batching) instead of forwarding raw samples unchanged; a scalable cloud tier built from managed AWS services rather than a single long-running server; a dashboard an operator could actually read at a glance; automated tests covering the arithmetic and edge cases that matter, not just the happy path; a continuous-integration pipeline that runs those tests on every push rather than trusting a developer to run them locally; and evidence the system runs somewhere real, an actual deployment to AWS rather than a description of one.

The remainder of this report is organised as follows. Section II sets out the architecture and the reasoning behind its more contested choices. Section III describes the software as implemented, including the Terraform module that provisions the entire cloud tier in one operation. Section IV reports the results of the automated test suite and what a live-browser check on the deployed dashboard found that a purely API-level check would not have. Section V narrates the real AWS deployment: the account it runs in, the defects a pre-deployment audit found and fixed, and the evidence gathered against the running system. Section VI concludes with an interpretation of those findings and a short reflection on building the project.

II. ARCHITECTURE

Each of the ten sensor containers represents one robot in one zone and publishes its five metrics on two independently configurable rates, one controlling how often a new reading is generated and the other how often a batch of readings is pushed to the fog gateway, so that noisy, frequent sampling can be decoupled from network chatter. The fog gateway is a plain Java HTTP server instead of a framework-managed service: it accepts ingest posts, buffers them per robot and per zone, and closes a window on a fixed interval, at which point it computes an aggregate and checks it against a set of alert rules. Battery telemetry receives particular attention in that rule set, because predictive battery monitoring for AMR fleets has been shown to catch degradation trends well before an outright failure, using continuous IIoT-style telemetry of exactly the kind this project collects instead of a periodic manual check [5]. The alert rules for motor temperature and position drift follow the same pattern: a threshold crossing is flagged at the window level, where the dashboard's detail panel can show the trend rather than an operator having to scroll a per-sample log.

On the cloud side, DynamoDB's own partition-key guidance sets a hard limit of 3,000 read-capacity units and 1,000 write-capacity units per partition, and recommends spreading writes so no single partition absorbs a disproportionate share of the fleet's traffic [6]. The table's key design, a sort key disambiguated by the window's end timestamp and the site identifier, follows that guidance by keying on the sensor/robot dimension instead of a single fleet-wide counter, so ten robots' windows land across the partition space rather than piling onto one key. SQS sits between the fog gateway and the ingestion function for the usual reason a queue separates a producer from a consumer that should not share a failure mode, but the batching decision built on top of it has a specific justification: SQS bills and rate-limits per request, and its batch API amortises that cost across up to ten messages instead of one request per message [7]. The relay publisher chunks a window's aggregates at exactly that ten-entry limit, called once per closed window instead of once per aggregate, the difference this project's own load test, described in Section IV, was built to demonstrate.

The overall pipeline shape (fog to queue to Lambda to DynamoDB, queried by a second Lambda behind API Gateway) is a widely used serverless pattern; what is distinctive here is

how the dashboard function models its routes. Each route is a distinct variant of a single sealed Java type, so the compiler can verify that every possible route is handled: adding a new route without handling it is a compile-time error rather than a not-found response discovered at runtime, a stronger guarantee than a lookup table or a reflection-driven router provides. The cost is a dispatch that is extended by hand for every new route instead of one populated declaratively, but the API surface here is five fixed routes unlikely to grow substantially, so the compile-time guarantee mattered more than declarative extensibility would have for a surface this size.

wrf-dashboard-api's health handler reasons about pipeline freshness by reading four things directly instead of trusting a single cached flag: whether the fog gateway's own health endpoint answers over HTTP, whether the SQS queue is reachable, whether the wrf-processor Lambda's own deployment state reports Active, and how old the most recent DynamoDB window is across every sensor type. That last check, the freshest-window age, is the one that actually detects a stalled pipeline: a queue and a Lambda can both report healthy while no new data has landed in over an hour, and only a timestamp comparison against the table itself catches that. The fog gateway's health and thresholds endpoints answer without any credential check, which reflects a scope decision specific to this deployment: they return only aggregate window state and configured threshold values, never a live per-robot reading, so what they expose is bounded to information an operator already sees rendered on the dashboard itself.

Two alternatives were weighed and set aside. Kinesis Data Streams would have kept an ordered, replayable record of every window per shard, which SQS's standard queue does not guarantee, but ordering was judged unnecessary here: the dashboard reads the latest window per sensor type from DynamoDB directly, not a stream of intermediate states, so Kinesis's stronger ordering guarantee would have added shard-management overhead without a matching need to spend it on. A single Lambda function handling both ingestion and dashboard queries was also considered and set aside, because ingestion is triggered by SQS on the fleet's own schedule while dashboard queries are triggered by whoever is looking at the browser: coupling the two would mean a slow dashboard query competing for the same concurrency budget as message processing, and splitting them into separate ingestion and dashboard functions keeps those two very different load profiles from interfering with each other.

III. IMPLEMENTATION

The entire pipeline is written in Java 17 against the plain Java HTTP server, with no web framework in any of the four modules: sensors, fog, processor, and dashboard. Each module builds and tests independently; the dashboard module additionally produces a second, Lambda-facing entry point alongside its local development server, so the same query and threshold logic serves both a laptop and a real API Gateway integration. Every AWS-facing test double is named for the

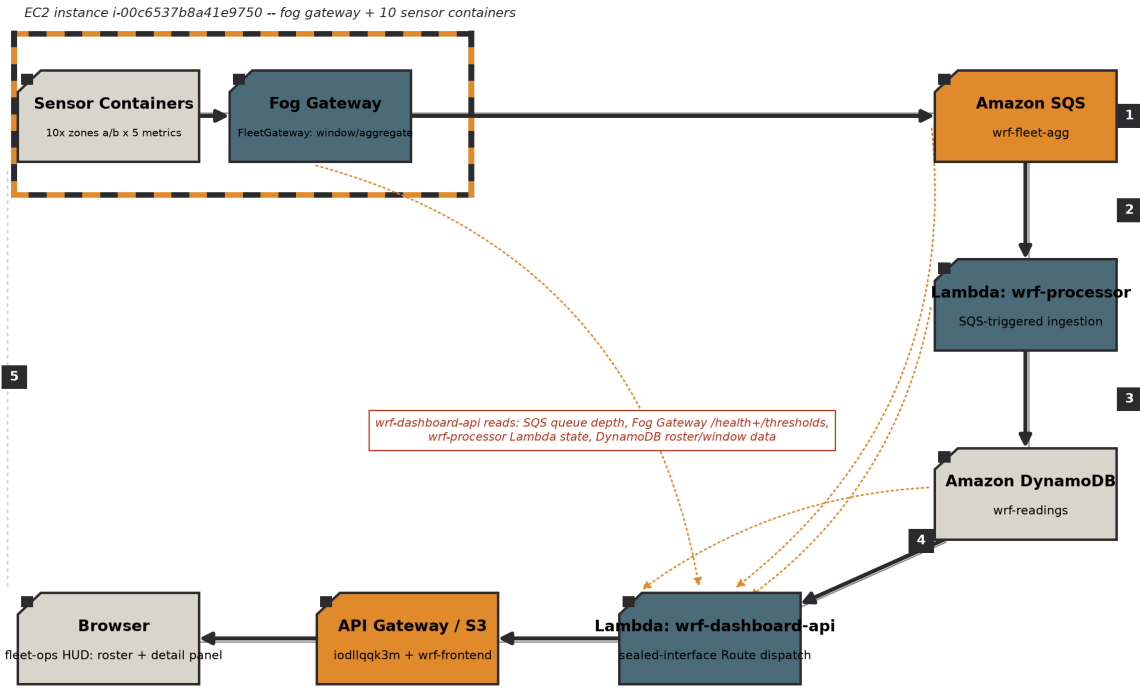


Fig. 1. Deployment topology. Sensors and Fog Gateway sit inside the EC2 warehouse-aisle loop’s top-left corner; the ingest path runs clockwise through SQS, wrf-processor and DynamoDB, and the query path runs counter-clockwise back through wrf-dashboard-api, API Gateway/S3 and the Browser. Three amber dotted spokes mark wrf-dashboard-api’s health/state reads (SQS queue depth, Fog Gateway, wrf-processor); the fourth, from DynamoDB, is drawn as a solid arrow since it doubles as the ingest path’s own final edge.

role it plays in a test rather than the client it mimics, so a reader can tell from the name what each one stands in for.

A GitHub Actions workflow runs on every push and pull request as two dependent jobs rather than one. The first, unit-tests, runs the Maven test goal in each of the four modules in turn (sensors, fog, backend/processor and backend/dashboard), so a failure in any one module’s own test suite fails the build before the second job starts. The second, integration, brings the full stack up with Docker Compose against LocalStack and runs the pipeline verification script against the running containers, exercising the same ingest-to-DynamoDB path this report verifies separately against real AWS in Section V, then tears the stack back down regardless of outcome so a failed run does not leave containers running on the runner.

Each sensor process simulates one robot’s one sensor type, using a bounded random walk to keep each metric’s drift in range instead of letting it wander arbitrarily between samples: position drift, for instance, is walked around a centre value rather than accumulating without limit, which matters because an unbounded drift value would eventually trip every threshold rule regardless of whether the robot is actually behaving abnormally. Ten containers run in total, one per zone/sensor-type combination across zone-a and zone-b, each independently configurable on its own sample and dispatch intervals.

The fog gateway ingests posted readings over HTTP, buffers them per robot and per zone so that one zone’s burst of traffic cannot delay another zone’s window from closing, computes a window aggregate when the window elapses, evaluates that

aggregate against the configured alert rules, and calls the relay publisher to send the result onward. A separate thresholds endpoint exposes the currently configured rule values so the dashboard can display them next to the readings they gate, instead of hardcoding a second copy of the same numbers into the frontend.

The processor’s Lambda entry point is invoked by an SQS event-source mapping; a pure transform step builds the table’s composite sort key and maps a batch of queue records into DynamoDB writes, with per-record failures tallied and returned as a partial batch-item failure response instead of one failure discarding the whole batch.

Every AWS resource this project runs on was provisioned by a single Terraform module, applied once against Goutham Uppu’s own Learner Lab account. The module’s 24 resources span every service group the deployment needs: the DynamoDB table; the SQS queue; both Lambda functions plus the event source mapping connecting the queue to wrf-processor; the full API Gateway REST API chain (a resource, a proxy method and integration at the root and at the greedy proxy path, a Lambda invoke permission, a deployment and a prod stage); the EC2 instance, its security group and its Elastic IP; and the two S3 buckets, the frontend bucket’s public-access block, bucket policy, website configuration and object upload, plus a provisioning step that ships the fog/sensor source tarball into the EC2 instance’s user-data script on first boot. An empirical study of infrastructure-as-code adoption across 44 companies found that the practice’s main payoff

TABLE I
AUTOMATED TEST SUITE BY MODULE (116 TESTS TOTAL)

Module	Tests	Focus
sensors	56	bounded random walk (53), dual-rate sample/dispatch loops (3)
fog	24	buffer, alert rules, exactly-at-boundary threshold checks
backend/processor	9	sort-key mapping, partial-batch-failure tallying
backend/dashboard	27	routes, Scan-pagination fix, Lambda dispatch tests

is reproducibility: the same declared state can be torn down and rebuilt without hand-run steps drifting between environments [8], and this project realises that payoff directly: rather than a sequence of individual AWS CLI commands run in order, one Terraform apply brought all 24 resources up with matching configuration, down to the dashboard Lambda’s fog-health and fog-thresholds environment variables being wired automatically to the Elastic IP Terraform itself allocated, with no manual copying of an IP address between commands. That convenience has a cost worth naming: the module couples the frontend upload and the EC2 bootstrap to the same apply, so a change touching only the dashboard’s static assets still triggers a plan against the entire stack, and a later iteration would benefit from splitting the frontend deploy into its own narrower Terraform workspace.

The project’s source is maintained at [GitHub repository URL]. The processor’s packaging script remains in the repository as the build step Terraform invokes to produce the shaded JAR the module uploads; it is no longer a deployment path in its own right, since Terraform now owns every AWS-side step that script used to perform by hand.

IV. EVALUATION

Each of the four Maven modules was run directly instead of trusted from a stated count: sensors’ 56 tests (53 covering bounded-drift behaviour across repeated walks, 3 covering the robot unit), fog’s 24 tests (the relay publisher, gateway, window aggregate, and alert-rule suites, the last of these covering exactly-at-boundary threshold behaviour rather than only clearly-over or clearly-under cases), the backend processor’s 9 tests (record mapping and handler behaviour, including partial-batch-failure tallying), and the backend dashboard’s 27 tests (the thresholds gateway, the pipeline checks, which specifically exercise the multi-page Scan-pagination fix described in Section V, the Lambda dispatch, the repository, and the local development server), for 116 tests total, all passing. No test suite touches a real AWS account or LocalStack; every AWS SDK v2 client interface used across these modules is backed by a hand-written fake instead of Mockito.

Scalability evidence beyond the unit level comes from a burst-test script, which drives 2,000 messages through 32 concurrent workers against the running stack. Its purpose is narrower than a full soak test: it demonstrates that the fog-to-queue path holds up under concurrent load from many

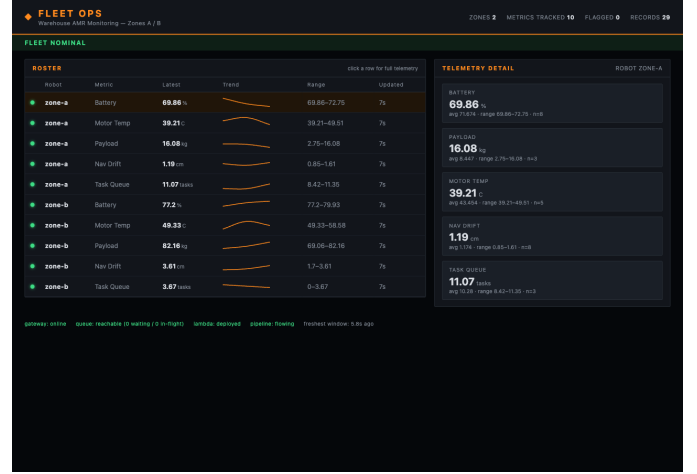


Fig. 2. The deployed fleet-ops dashboard at its S3 URL, backed by wrf-dashboard-api through API Gateway: the two-zone robot roster with inline sparklines and LED status indicators, the per-robot telemetry detail panel, and all four pipeline health checks reporting healthy.

simulated robots at once, independent of whatever cadence the ten containers themselves are configured to run at.

The deployed dashboard was checked two ways. curl against the health, backend-stats, fleet and thresholds endpoints confirmed the API Gateway integration answers correctly, and successive backend-stats polls showed the DynamoDB item count increasing over time, consistent with sensor data actively arriving instead of a table that had stopped updating. curl alone cannot confirm that a browser loading the dashboard’s S3 URL can actually reach that API, parse the response and render it: a CORS misconfiguration or a broken static-asset path returns a perfectly valid response to curl while leaving the dashboard blank in every browser that loads it, a failure mode that is easy to miss precisely because every API-level check passes. A Playwright session against the live URL confirmed the fleet-ops HUD (the roster table with its inline sparklines and LED status indicators, and the detail panel beneath it) renders real, changing sensor data with no console error beyond a single missing-favicon 404, which has no bearing on the pipeline. Fig. 2 reproduces the view that session captured against the live deployment.

V. DEPLOYMENT

Warehouse Robotics Fleet Monitoring runs in Goutham Uppu’s own AWS Academy Learner Lab account, 789399341650, region us-east-1, under the account’s session role, reusing the shared lab role and instance profile instead of any role created specifically for this project: the account cannot create new IAM roles or users, a standard constraint of AWS Academy Learner Lab accounts.

A pre-deployment code audit examined the item-count method the backend-stats endpoint calls to report how many items sit in the readings table, and found it issued a single scan and returned that scan’s count directly: correct only while the table holds fewer items than one scan page returns, and wrong

once it grows past that without any indication in the returned number. The fix walks every page of the scan, following the pagination cursor from one page to the next and summing the per-page counts, so the reported total stays correct regardless of table size.

The same audit found the fog gateway relaying each window’s aggregates to SQS one send at a time. The relay publisher now chunks a window’s payloads into groups of up to ten and issues one batch send per chunk, called once per closed window instead of once per aggregate.

The most consequential defect was in credential handling, present in all three AWS-facing classes: the processor’s client builder, the local dashboard server’s client builder, and the relay publisher’s constructor all built a static test-credentials provider unconditionally, instead of only when an explicit LocalStack endpoint was configured. In the two client builders that would have meant a real Lambda invocation quietly overriding its own execution-role credentials with a pair AWS would reject outright. In the relay publisher the same bug was worse: the endpoint override was also applied unconditionally in the original code, and the endpoint value is null outside a LocalStack environment, so the fog node would have thrown a null-pointer exception and failed to start at all the first time it ran against real AWS, not merely authenticated incorrectly. All three were fixed to gate on the endpoint variable’s presence, matching the pattern the codebase already used correctly in one place.

readme.txt originally described the pipeline’s architectural conventions (the SQS-to-Lambda-to-DynamoDB shape, the window-end/site-id sort-key scheme, the dashboard health-check pattern and the dual-rate sensor knobs) in language that read as though those conventions were invented for this project rather than established, published patterns. That description was corrected to credit the underlying patterns plainly, while still identifying what is original to this project: the domain-specific sensor profiles and alert thresholds, and the entire dashboard’s dark orange-and-black HUD, roster table and detail panel.

With those four defects corrected, the stack was provisioned in a single Terraform apply against the confirmed account (24 resources, described in Section III, with zero manual AWS CLI steps in between). The result: DynamoDB table wrf-readings, SQS queue wrf-fleet-agg, Lambda functions wrf-processor and wrf-dashboard-api behind API Gateway REST API iodllqqk3m, EC2 instance i-00c6537b8a41e9750 running the fog gateway and all ten sensor containers behind Elastic IP 3.211.126.248, and S3 buckets wrf-frontend-789399341650 and wrf-deploy-789399341650. Verification went beyond the application’s own self-report: the health endpoint was polled directly and returned all four fields true, backend-stats showed the DynamoDB item count climbing across successive polls, and the dashboard was opened in a Playwright-driven browser session at its S3 URL, where it rendered the fleet roster and detail panel with live data and no console error beyond the one harmless missing-favicon 404 already noted in Section IV.

TABLE II
LIVE AWS RESOURCES (ACCOUNT 789399341650, US-EAST-1)

Resource	Identifier	Role
DynamoDB table	wrf-readings	stores closed-window aggregates
SQS queue	wrf-fleet-agg	decouples fog gateway from wrf-processor
Lambda	wrf-processor	SQS-triggered ingestion into DynamoDB
Lambda	wrf-dashboard-api	sealed-type route dispatch behind API Gateway
API Gateway REST API	iodllqqk3m	public entry point for the dashboard API
EC2 instance	i-00c6537b8a41e9750	fog gateway + 10 sensor containers
Elastic IP	3.211.126.248	fixed address for the EC2 instance
S3 buckets	wrf-frontend-789399341650 / wrf-deploy-789399341650	static dashboard frontend / deploy staging

VI. CONCLUSION

Warehouse Robotics Fleet Monitoring’s central finding is less about any single component than about where correctness actually broke down: every one of the four defects a pre-deployment audit found lived at a boundary the local test suite could not see across; a Scan response’s per-page count versus the whole table’s count, a single SQS publish versus a batched one, a credential provider built for LocalStack versus one built for a real Lambda execution role, and a documentation claim versus what the code underneath it actually rests on. None of the 116 unit tests that passed before the audit began would have caught any of the four, because none of them exercised more than one Scan page, more than one publish call, a real endpoint-absent code path, or the readme’s own prose. That is the practical argument for treating a code audit against real service semantics as a distinct step from running an existing test suite, not a substitute for one.

The Terraform module is the other substantive outcome: a CLI-scripted deployment assembles a cloud tier by running AWS commands in sequence, copying an Elastic IP’s value from one command’s output into the next command’s environment variable by hand. Collapsing that into one declared module and one apply removes the specific failure mode where a manually copied IP address is stale or mistyped, and gives the deployment a single state file recording what was actually created.

Working through this project meant learning to distrust a green test suite as proof of correctness against a real cloud account: the pagination and credential defects both passed every existing test cleanly, because the tests were written against the same mental model that produced the bug. Writing the item-count fix to page through the whole scan rather than trusting a single response also made the hidden cost of assuming small data concrete, a useful exercise on its own terms, separate from the bug it fixed. The Terraform module

was the part that took longest to get right, mainly because the EC2 user-data script's dependency on the S3 upload finishing first was easy to get backwards, and debugging a failed Terraform apply against a session-limited Learner Lab account meant working faster than a full plan-apply cycle really allows, a constraint that shaped how much could be verified per attempt rather than what was ultimately built.

Two things remain open. The Terraform module's coupling between the frontend upload and the EC2 bootstrap, noted in Section III, would be worth splitting once the deployment needs to change more often than it does today. A live walk-through demonstration remains to be delivered separately from this report; everything reported here is what the code, the test suite, and the live AWS account show today, independently checked rather than taken from the project's own status claims.

REFERENCES

- [1] A. Souto, P. A. Prates, A. Lourenco, M. S. Al Maamari, F. Marques, D. Taranta, L. Doo, R. Mendonca, and J. Barata, "Fleet Management System for Autonomous Mobile Robots in Secure Shop-floor Environments," in *Proc. 2021 IEEE 30th Int. Symp. on Industrial Electronics (ISIE)*, Kyoto, Japan, Jun. 2021, doi: 10.1109/ISIE45552.2021.9576269.
- [2] T. Lackner, J. Hermann, C. Kuhn, and D. Palm, "Review of Autonomous Mobile Robots in Intralogistics: State-of-the-Art, Limitations and Research Gaps," *Procedia CIRP*, vol. 130, pp. 930–935, 2024.
- [3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. 1st Ed. of the MCC Workshop on Mobile Cloud Computing (ACM SIGCOMM MCC '12)*, Helsinki, Finland, Aug. 2012, pp. 13–16, doi: 10.1145/2342509.2342513.
- [4] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-145, Sep. 2011, doi: 10.6028/NIST.SP.800-145.
- [5] K. Krot, G. Iskierka, B. Poskart, and A. Gola, "Predictive Monitoring System for Autonomous Mobile Robots Battery Management Using the Industrial Internet of Things Technology," *Materials*, vol. 15, no. 19, p. 6561, Sep. 2022, doi: 10.3390/ma15196561.
- [6] Amazon Web Services, "Best Practices for Designing and Using Partition Keys Effectively," Amazon DynamoDB Developer Guide, [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html>.
- [7] Amazon Web Services, "Increasing Throughput Using Horizontal Scaling and Action Batching with Amazon SQS," Amazon Simple Queue Service Developer Guide, [Online]. Available: <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-throughput-horizontal-scaling-and-batching.html>.
- [8] M. Guerriero, M. Garriga, D. A. Tamburri, and F. Palomba, "Adoption, Support, and Challenges of Infrastructure-as-Code: Insights from Industry," in *Proc. 2019 IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, Cleveland, OH, USA, 2019, pp. 580–589, doi: 10.1109/ICSME.2019.00092.